

当 Ant Design 遇上 CSS 变量

当 Ant Design 遇上 CSS 变量

Ant Design 5.0 的痛点

antd 可以通过 ConfigProvider 对主题 token 进行定制，同样支持嵌套主题，并且嵌套的主题 token 会继承上一层已经修改的部分。从这一点上来讲，antd 的主题能力已经达到了 5 个大版本以来的巅峰。

当然我写这篇文章并不是为了把 antd 5.0 再拉出来夸一遍的，这件事在 5.0 发布的时候已经做过了。自 5.0 发布至今已经过去了快一年，其间也充斥着社区的各种质疑与需求，这些也都是 antd 目前面临的痛点与优化方向。

CSS 产物体积

这一点的主要影响的为 SSR 场景用户。

antd 其实切换到 cssinjs 的时候已经默认实现了按需引入 css 的能力，不需要依靠 babel-plugin-import 就能够实现自动的 tree-shaking。但是与 MUI 或者 Mantine 这些一开始就已经使用 cssinjs 的组件库相比，antd 使用 cssinjs 其实算是半路出家，也因此有一些历史债务是无法去除的，在 cssinjs 的使用方面也与其他组件库不同：**antd 的 cssinjs 并不跟随组件本身的 props 改变。**

在 MUI 与 Mantine 6.x 中，配置了不同 props 的组件所生成的 css 其实是不一样的，而这些样式都会放在一个 hash class 里面。是的没错，MUI 的那些具名的 className 比如 xxx-focused 都是空壳，是方便用户进行魔改的。这样做的好处是会大大减少样式文件的大小，比如一个页面只使用了 outlined 变体的 Button，那么最终的样式里就不会存在 filled 变体的样式。

图片说明：image

而在 antd 5.0 中，只要使用了某个组件，antd 就会自动引入有关这个组件的所有样式——不管有没有使用过。这样做有两个原因：

1. antd 从 4.x 到 5.x 进行改造时并没有改变样式的组织方式，任然是和 4.x 一样通过 class 的组合来实现不同的样式效果。
2. 减少动态生成样式的次数。我们以组件为维度制定了缓存策略，同一个组件只会插入一次样式，这样会减少 cssinjs 在序列化 css 时的性能损耗。

可以看出传统 cssinjs 和 antd 各自的优点其实反过来也是对方的缺点。antd 的 css 产物大小因此变得非常庞大，这一点在 SSR 场景下需要内联 style 标签时就显得格外碍眼。

主题切换

在 cssinjs 的组件库中切换亮暗色主题时我们通常会发现两个问题：

1. 切换主题时有卡顿感
2. 静态站点切换到暗色后刷新，总是会从亮色再变回暗色

这两个问题其实 antd 也中招了，其原因归结起来其实还是由于 cssinjs 运行时生成样式的特性。前者卡顿是因为需要进行一轮新的 css 序列化；后者不能无缝刷新是因为静态站点不能保留切换后主题的风格。

这两个问题在 CSS 变量的主题系统下就不存在，典型页面就是 react.dev。原理其实也很简单：

1. 修改 CSS 变量不需要重新序列化 css，省去了这一性能消耗
2. 在页面渲染之前 CSS 变量就可以进行注入，即通过 body 下的 script 来阻塞渲染，从而避免了渲染不必要的风格。

如何破局？

结合 Mantine 7.0 的案例来看，似乎以 CSS 变量为基础的主题系统/风格引擎拥有更好的用户体验。但是对于 cssinjs 我们也有不能割舍的理由——我们并不希望像 Mantine 那样舍弃更加灵活的主题能力。也就是说我们希望拥有 CSS 变量体积小、切换快的特性，又希望保留 cssinjs 的主题嵌套、多主题并存的能力。

那么我们自然而然的就想到能不能将 cssinjs 与 CSS 变量结合到一起？毕竟他们看起来并不是完全互斥的。

Ant Design 与 CSS 变量

antd 不是第一次与 CSS 变量合作了，早在 4.x 时代就已经有了一套基于 CSS 变量的 CSS 文件。4.x 中 antd 的主题主要通过 less 变量来实现，将 less 变量赋值为 CSS 变量名，然后在其他地方给这些 CSS 变量赋值，这就是 antd 4.x CSS 变量主题的能力。

antd 5.0 的主题能力其实由 4.x 的进化而来，同样拥有一套主题 token 来进行主题定制。这是目前的主题系统的处理过程，其中我们会基于主题 Token 的值来计算一个唯一的 hash 变量，从而保证主题之间相互隔离：

图片说明：image

:where 选择器不会提高整体权重，所以很适合用来做主题隔离

CSS 变量映射

所以很自然地，我们也想到了将所有 token 用 CSS 变量填充的方案，于是这篇 RFC 便诞生了：RFC CSS variable theme of antd

在这篇 RFC 中，我们引入了一个设想：将所有 token 都映射到 CSS 变量，并用 CSS 变量来填充 token 的值。如此一来 antd 对于主题的 hash 计算就会固定，因为每个 token 的值都不会再变了。这时我们就会得到一份稳定的 HTML，想要切换主题时只需要替换对应的 CSS 变量，而不需要经历冗长的 cssinjs 序列化过程。

图片说明：image

现在我们融合 cssinjs 和 CSS 变量的方案初具雏形了。这个方案中，其实 CSS 变量插入的环节被排除在了整个主题的生命周期之外，因为我们只关心 token 和 CSS 变量之间的相互替换。只要 antd 将 CSS 变量运用到了组件样式的各个角落，我们自然就可以在此之上进行基于 CSS 变量的主题构筑。

如上图所示，控制整个主题的 CSS 变量最终被放在了 :root 选择器下，这也就意味着我们可以在任何时机修改这一份 CSS 变量：可以是编译时，也可以在浏览器环境中。但是另一方面，将 CSS 变量放置在 :root 下也意味着这将会是一份影响范围是整个 document 的主题，我们无法进行局部主题的调整。

那么现在问题就变成了：我们能否让 CSS 变量局部生效？

CSS 变量隔离

答案是可以的。还记得我们在 5.0 引入的 hash 吗？这就是解决这个问题的关键。

通过将 CSS 变量约束在 hash 的类选择器下，我们就可以让这批 CSS 变量只对该主题下的组件生效。同时我们也可以利用上下文所提供的主题来实时地生产 CSS 变量：把当前主题配置的 token 值直接转化为 CSS 变量，组合上当前的 hash 值就得到了一份完整的样式。

图片说明：image

看起来很完美，利用现有的主题特性实现了 CSS 变量在主题之间隔离的功能。但是实际上到目前为止这个方案有一个巨大的缺陷，而这在上文中也提到过：为了保证 HTML 的稳定，计算 hash 时我们实际上是使用的映射到 CSS 变量后的 token 值，也就是类似 var(--color-primary) 的值；而这些值是不会变的，因为我们不会去刻意修改 CSS 变量与 token 间的映射关系，**所以这也会导致 hash 是一个固定值。**

考虑嵌套的场景：

Button 1

Button 2

在 antd 5.0 当前的主题系统下，这两个 Button 所对应的 hash 值是不一样的，所以他们的样式互不影响，这就是 hash 在主题隔离的作用。

但是在 CSS 变量的方案下，这两处 token 修改并不会实际影响 hash 的计算，所以会导致主题隔离失效，两个 colorPrimary 会被放置在相同的 hash 下，导致相互覆盖。在保证主题隔离的前提下，我们需要 Button 1 和 Button 2 的 hash 值不一样，但这又明显是矛盾的，产生了一个新的问题。

回过头来思考一下我们采用 CSS 变量的初衷：****实现更快的主题切换，降低主题切换的性能消耗。****可以发现重点其实在“切换”上。

对于嵌套主题或者平行主题来说，他们的侧重点并不是“切换”而是“隔离”，所以需要各异且稳定的 hash，在大多数场景下他们的 HTML 也是稳定的。

对于切换主题来说，重点就来到了“切换”。我们所期望的是在固定 hash（即稳定的 HTML）的情况下利用 CSS 变量实现快速、高性能的主题切换，这一点与主题隔离并不冲突，所以我们仍然需要不同的 hash 来 CSS 变量隔离的局面。在此之上，再根据用户的需求生产各个 hash 下不同的主题对应的 CSS 变量样式。

这说起来有些抽象，我们来用代码解释一下想要的效果。同样是上面举出的嵌套主题的场景，我们最终希望得到的 CSS 文件应该包含这些：

```
:where(.css-hash1).ant-btn {  
  background-color: var(--color-primary);  
}
```

```
:where(.css-hash2).ant-btn {  
  background-color: var(--color-primary);  
}
```

```
.css-hash1 {  
  --color-primary: blue;  
}
```

```
.css-hash2 {
```

```
--color-primary: green;
}
```

这样的产物是符合预期的，它将 CSS 变量与 hash 结合到了一起。但实际上这样我们应该稍微调整一下我们的思路，将 hash 计算回归到 token 原本的值上，或者直接利用 CSS 变量来计算 hash：

图片说明：image

CSS 变量切换

到目前为止，我们通过 hash 将主题隔离与 CSS 变量融合到了一起，满足了一下我们既要又要的野心。但是 hash 其实还有一个问题——它是动态计算的，用户无法提前得知 hash 值到底是什么，也就无法实现直接用 js 操作修改 CSS 变量。

但是办法总是有的。我们除了直接利用 js 修改 CSS 变量之外，还可以利用 CSS 选择器来实现不同场景下 CSS 变量的切换，这就要求我们提前把不同主题的 CSS 变量同时生成好：

```
/* 利用自定义的主题 class 作为嵌套选择器 */
.light .css-hash {
  --color-text: rgba(0, 0, 0, 0.85);
}

.dark .css-hash {
  --color-text: rgba(255, 255, 255, 0.88);
}
```

此时我们只需要切换包裹在组件外层的 DOM 上的 class，就可以轻易实现主题的切换。这个外层 DOM 可以是 html，可以是 body，也可以是应用中的某一层元素——这由用户自由决定。

问题总是一环套一环的。我们之前提到 hash 还是正常通过 token 计算的，但是涉及到动态切换时我们又希望 hash 是不变的。所以想上文中列出的两套主题，他们的 hash 应该是一致的，尽管他们对应的 CSS 变量并不相同。于是我们需要在 hash 的计算方法上做一些文章。

图片说明：image

实际上可以发现我们在套用 CSS 变量之后，我们一直在做的一件事情就是把“动态”变为“静态”，极力去避免使用 js 动态修改 token 的情况发生，因为这一定会唤起 js 的计算逻辑从而拖慢网页性能。相对的，我们利用 CSS 变量可提前编译的静态能力，将运行

时会产生变化的东西一步步缩小范围，最终变成只需要修改一个类名或者 DOM 属性就可以做到高性能的主题切换。

最后别忘了 token 是可以通过 context 进行传递的，在嵌套主题时，当前的 token 会继承来自上下文的 token 值进行覆盖。根据上文中的描述，我们需要在主题中提供复数的 token，他们分别拥有不同的主题名。如果在嵌套的主题中多层使用了多主题，那么其计算量其实是乘算的。举个例子：

图片说明：image

这会成为一个潜在的负担。

CSS 变量动态主题

考虑这样的场景：用户可以在网页中任意修改主题色以及文字字号，以符合自己的喜好。这个场景与上文所研究的就不适配了，原因是我们之前的研究建立在有一整套已知的主题（如暗色主题）上，但用户可以随意修改某个变量值的时候我们就不能够提前对主题进行构建，只能依赖运行时的能力进行修改。这种场景可以称为是真正的“动态主题”。

如同上文所说，实现基于 CSS 变量的动态主题有两个阻碍：

1. hash 值变化会导致组件和 DOM 重新渲染；
2. 用户无法提前得知 hash 值

而针对这两点分别有两种解决方案。

随机 hash

对于前者，我们需要解决的问题明显是 token 变化带来的动态 hash 值。hash 值存在的原因有两个，样式隔离和缓存。样式隔离需要的是每个主题对应不一样的 hash，缓存需要的是每个主题对应唯一的 hash。在 CSS 动态主题的场景下，我们可以抛弃缓存这一特性，因为我们已经将所有的 token 替换为了 CSS 变量，样式本身并不会改变，每次改变 token 只会改变插入的 CSS 变量，这时**序列化 CSS 带来的性能消耗已经被大幅减小了**。因此我们完全可以采用随机 hash 来保证样式隔离。

图片说明：image

这时我们可以发现用户想要使用动态主题时只要像之前一样在 ConfigProvider 里修改 token 就可以了，不会产生任何使用上的变化。而额外的一点就是，这里我们省去了计算 hash 带来的性能损耗，但相对的我们可能会生产出一模一样的两套 CSS 变量而他们的 hash 并不一致。这一点具体会带来正面还是负面的影响还得具体看用户的使用方法。

自定义 hash

上述的方案其实已经可以解决大部分场景下的问题，但我们回过头看还有一个从一开始就提到的问题：FOUC 问题。在一个已经静态编译的网页上，所有利用 react 生命周期实现的主题变化都不能在第一时间反应到用户的浏览器上。我们必须提供给用户能够在浏览器渲染页面前，就能够修改主题的能力。

这种能力的实现方式其实就是能够让用户在脚本中替换 CSS 变量。我们在上面提到了一种方法是在脚本中直接替换 html 上的 class 或者属性来套用已经静态编译完成的 CSS 变量，这对于动态主题并不适用。所以用户就需要在脚本中直接修改 CSS 变量的值，所以就引入了“需要提前得知 hash 值”这个问题，因为如果 hash 值完全随机或者根据 token 计算，用户就无法在 react 生命周期之外使用这个 hash 值；或者说很难把这个 hash 值存下来，供在 react 生命周期之外的 script 使用。

那么换句话讲，只要用户知道 hash 值是不是就可以了呢？

所以我们允许用户 diy hash 值，用户也不需要担心因为自定义的原因导致样式隔离失效——我们很容易就可以检测出用户在应用中使用了相同的 hash 值。如此一来用户只需要按照 antd 的格式来覆盖 hashed CSS 变量就可以了——我们可以提供一个工厂函数来帮助用户生产 CSS 变量样式。

图片说明：image

再见了 hash

不再主动计算 hash 后，我们心中冒出了两个问题：

1. hash 它还是 hash 吗？
2. 我们还需要在样式上添加 hash 吗？

第一个问题其实有点哲学，从发展历程来讲，它就是 hash。但是他已经不再需要计算了，所以就是一个纯粹的随机值或者用户自定义的字符串。

第二个问题很重要。在所有 token 都被替换的现在，不同主题下的样式已经不会再有任何区别了，hash 起到的隔离作用也不再重要。我们仍然会在 DOM 上添加 hash class 作为主题的 scope，它会直接影响组件所采用的 CSS 变量源于何处。但是样式并不关心这些，所以我们再进行一次优化：

图片说明：image

样式居然可以单独存在了！这意味着不同的主题可以共用同一份样式——我们不再需要重复生成这些样式了。

当然还需要考虑微应用场景，不同版本的 antd 之间虽然 hash 是隔离的，但是样式失去了 hash 之后也就失去了作用域，不同版本的 antd 会相互污染，所以我们仍会提供对应用整体的样式打上 hash 的能力——这就是应用级别的 hash，与主题 hash 不同，整个应用仍然可以复用同一份样式。